

React Native Mobile App Development

5-Day Beginner React Native Course

Table of Contents

1. [Course Overview](#)
2. [Day 1: React Native Foundations & Environment Setup](#)
3. [Day 2: Core Components, Styling & Layout](#)
4. [Day 3: Lists, Forms & User Interactions](#)
5. [Day 4: Navigation & State Management](#)
6. [Day 5: Native Features, Debugging & Publishing](#)
7. [Final Assessment](#)
8. [React Native Quick Reference](#)
9. [Glossary](#)

Course Overview

What is React Native?

React Native is an open-source framework that allows developers to build native mobile apps for Android, iOS, and other platforms using React and JavaScript . Instead of writing separate code for each platform, React Native lets you write once and deploy everywhere, using the same native platform APIs as other apps .

Key Benefits:

- **Cross-platform development:** Single codebase for Android and iOS
- **Native performance:** Code compiles to native components, delivering near-native performance
- **Hot reloading:** See changes in real-time during development
- **Large ecosystem:** Rich library of community components and tools
- **Meta supported:** Maintained by Meta with active community contributions

Prerequisites

- Basic understanding of JavaScript (variables, functions, arrays, objects)
- Familiarity with React concepts (components, props, state)

- Node.js installed (version 18+)

Course Materials

- Computer with Node.js 18+
- VS Code with React Native extensions
- Expo CLI or React Native CLI
- Android Studio (for Android emulator) or Xcode (for iOS simulator - Mac only)

How React Native Works

React Native bridges JavaScript code to native platform components . Instead of using web components like `<div>` and `<span>` , React Native uses native components like `<View>` and `<Text>` that map directly to platform UI building blocks .

React Native Architecture:

JavaScript Code → React Native Bridge → Native Components (Android/iOS)

Platform Mapping:

React Native Component	Android View	iOS View	Web Analog
<code><View></code>	<code><ViewGroup></code>	<code><UIView></code>	<code><div></code>
<code><Text></code>	<code><TextView></code>	<code><UITextView></code>	<code><p></code>
<code><Image></code>	<code><ImageView></code>	<code><UIImageView></code>	<code></code>
<code><ScrollView></code>	<code><ScrollView></code>	<code><UIScrollView></code>	<code><div></code>
<code><TextInput></code>	<code><EditText></code>	<code><UITextField></code>	<code><input></code>

Environment Setup

Option 1: Expo (Recommended for Beginners)

Expo is a production-grade React Native Framework that provides developer tooling, file-based routing, and a standard library of native modules .

```
# Create a new Expo project
npx create-expo-app@latest MyFirstApp
```

```
# Navigate to project
cd MyFirstApp

# Start the development server
npx expo start
```

Expo Go App: Download Expo Go on your physical device to preview your app instantly .

Option 2: React Native CLI (Advanced)

For more control over native code, you can use the React Native CLI:

```
# Install React Native CLI
npm install -g react-native-cli

# Create a new project
npx react-native init MyFirstApp

# Run on Android
npx react-native run-android

# Run on iOS (Mac only)
npx react-native run-ios
```

Recommendation: The official React Native documentation recommends using a Framework like Expo for new apps . You'll be able to spend time writing your app instead of writing an entire framework yourself .

Day 1: React Native Foundations & Environment Setup

Learning Objectives

By the end of Day 1, you will be able to:

- Set up a React Native development environment
- Understand React Native's architecture and core concepts
- Create your first React Native app
- Understand the difference between Expo and React Native CLI
- Navigate the project structure

1.1 What is React Native?

The Philosophy

React Native brings the "learn once, write anywhere" philosophy to mobile development . It allows developers who know React to create native apps, while native developers can use React Native to gain parity between platforms by writing common features once .

Use Cases

Companies using React Native include:

- Facebook (Meta)
- Instagram
- Airbnb
- Thousands of companies worldwide

React Native vs Other Frameworks

Feature	React Native	Flutter	Native (iOS/Android)
Language	JavaScript	Dart	Swift/Kotlin
Code Reuse	High	High	Low
Performance	Native	Native	Native
Learning Curve	Moderate	Moderate	Steep
Ecosystem	Mature	Growing	Platform-specific

1.2 React Native vs Expo

Expo Framework

Expo is a Framework built on top of React Native that provides :

Benefits:

- File-based routing
- Standard library of native modules
- Developer tools (Expo Go for instant preview)
- Simplified build process
- Free and open source

When to Use Expo:

- Starting a new app
- Building quickly without native code management
- Learning React Native

Expo Application Services (EAS): Optional services for build, submission, and updates .

React Native CLI

Benefits:

- Full control over native code
- Can add any native library
- More flexibility

When to Use React Native CLI:

- Need to write custom native modules
- Existing native codebase
- Specific requirements not served by Expo

1.3 Project Structure

Expo Project Structure

```
MyFirstApp/
├── .expo/                # Expo configuration
├── .gitignore
├── app.json              # App configuration
├── assets/               # Images, fonts, etc.
├── babel.config.js
├── node_modules/
├── package.json
├── src/                  # Source code (common pattern)
│   ├── components/      # Reusable components
│   ├── screens/          # Screen components
│   ├── navigation/       # Navigation setup
│   ├── hooks/            # Custom hooks
│   └── utils/            # Utilities
└── tsconfig.json         # TypeScript configuration
```

Key Files

File	Purpose
app.json	App name, icon, splash screen configuration
package.json	Dependencies and scripts
App.js or App.tsx	Entry point of your app

1.4 Your First React Native App

Hello World

```
// App.js
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Text style={styles.text}>Hello, World!</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    backgroundColor: '#f5f5f5',
  },
  text: {
    fontSize: 24,
    color: '#333',
  },
});
```

What's Happening Here?

1. **Imports:** We import React and core components (View , Text) from react-native
2. **Component:** App is a functional component returning JSX
3. **View:** A container that supports layout with flexbox
4. **Text:** Displays and styles strings of text
5. **StyleSheet:** Creates styles similar to CSS but with camelCase properties

Understanding JSX

JSX lets you write markup language inside JavaScript . Instead of web things like `<div>` or `<span>` , React Native uses native components like `<View>` and `<Text>` .

```
// This is JSX
<View>
  <Text>Hello World!</Text>
</View>

// React Native transforms this to native components
// On Android: ViewGroup containing TextView
// On iOS: UIView containing UITextView
```

1.5 Core Components Overview

Essential Components

Component	Purpose	Example
<View>	Container for other components	Wrapper for layout
<Text>	Display text	Hello World
<Image>	Display images	source={require('./img.png')}
<ScrollView>	Scrollable container	Long content
<TextInput>	User text input	Forms
<Button>	Touchable button	Actions
<FlatList>	Efficient list rendering	Large data sets

Component Structure

```
import { View, Text, Image, ScrollView, TextInput, Button, FlatList } from 'react-nati
```

Day 1 Exercises

Exercise 1.1: Hello World

Instructions: Create a new Expo project and build a "Hello World" app.

```
# Create project
npx create-expo-app@latest HelloWorld

# Navigate to project
cd HelloWorld

# Start development server
npx expo start
```

Tasks:

1. Open App.js
2. Replace the content with the Hello World example above
3. Run on Expo Go or emulator
4. Customize the text color and background

Exercise 1.2: Understanding Components

Instructions: Create a simple app with multiple components.

```
import React from 'react';
import { View, Text, Image, StyleSheet } from 'react-native';

export default function App() {
  return (
    <View style={styles.container}>
      <Image
        source={{ uri: 'https://reactnative.dev/img/header_logo.svg' }}
        style={styles.logo}
      />
      <Text style={styles.title}>Welcome to React Native!</Text>
      <Text style={styles.subtitle}>Learn once, write anywhere</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: 'center',
    alignItems: 'center',
    padding: 20,
    backgroundColor: '#ffffff',
  },
  logo: {
```

```
    width: 200,
    height: 200,
    marginBottom: 20,
  },
  title: {
    fontSize: 24,
    fontWeight: 'bold',
    marginBottom: 10,
  },
  subtitle: {
    fontSize: 16,
    color: '#666',
  },
});
```

Exercise 1.3: Project Structure

Instructions: Create a new Expo project with a custom structure.

```
# Create project
npx create-expo-app@latest MyApp

# Navigate to project
cd MyApp

# Create src directory
mkdir src
mkdir src/components
mkdir src/screens
mkdir src/assets
```

Create a reusable component:

```
// src/components/Header.js
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

export default function Header({ title }) {
  return (
    <View style={styles.header}>
      <Text style={styles.title}>{title}</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  header: {
    padding: 20,
```

```
    backgroundColor: '#007AFF',
    alignItems: 'center',
  },
  title: {
    fontSize: 20,
    fontWeight: 'bold',
    color: '#fff',
  },
});
```

Day 1 Summary

Key Takeaways

1. **React Native:** Framework for building native mobile apps with JavaScript and React
2. **Expo:** Recommended Framework for starting new React Native apps
3. **Components:** Building blocks like `View`, `Text`, `Image` that map to native components
4. **JSX:** JavaScript syntax for writing markup inside code
5. **StyleSheet:** Create styles with camelCase properties

Quiz Yourself

1. What is React Native?
2. What is the difference between Expo and React Native CLI?
3. What is the purpose of the `View` component?
4. What does the `Text` component do?
5. What is JSX?

Day 2: Core Components, Styling & Layout

Learning Objectives

By the end of Day 2, you will be able to:

- Work with core React Native components
- Apply styles using `StyleSheet`
- Use Flexbox for layout
- Create responsive layouts
- Handle touch events

2.1 Core Components Deep Dive

View Component

The `View` is the most fundamental component in React Native - a container that supports layout with flexbox, style, touch handling, and accessibility controls .

```
import { View } from 'react-native';

<View style={styles.container}>
  {/* Children go here */}
</View>
```

Text Component

The `Text` component displays and styles strings of text and can handle touch events .

```
import { Text } from 'react-native';

<Text style={styles.heading}>Hello World</Text>
<Text numberOfLines={2}>This text will be truncated after two lines</Text>
```

Image Component

The `Image` component displays different types of images .

```
import { Image } from 'react-native';

// Local image
<Image source={require('./assets/logo.png')} style={styles.logo} />

// Remote image
<Image source={{ uri: 'https://example.com/image.jpg' }} style={styles.logo} />

// With resize mode
<Image
  source={{ uri: 'https://example.com/image.jpg' }}
  style={styles.background}
  resizeMode="cover"
/>
```

ScrollView Component

`ScrollView` provides a generic scrolling container that can contain multiple components and views .

```
import { ScrollView } from 'react-native';

<ScrollView
  horizontal={false}
  showsVerticalScrollIndicator={false}
>
  <View style={styles.item} />
  <View style={styles.item} />
  <View style={styles.item} />
</ScrollView>
```

2.2 Styling with StyleSheet

Basic Styling

```
import { StyleSheet } from 'react-native';

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: '#fff',
    padding: 20,
  },
  heading: {
    fontSize: 24,
    fontWeight: 'bold',
    color: '#333',
    marginBottom: 10,
  },
  paragraph: {
    fontSize: 16,
    color: '#666',
    lineHeight: 24,
  },
});
```

Common Style Properties

Category	Properties	Example
Layout	<code>flex</code> , <code>flexDirection</code> , <code>justifyContent</code> , <code>alignItems</code>	<code>flex: 1</code>

Category	Properties	Example
Dimensions	width , height , padding , margin	width: '100%'
Text	fontSize , fontWeight , color , textAlign	fontSize: 16
Background	backgroundColor	backgroundColor: '#f5f5f5'
Border	borderWidth , borderColor , borderRadius	borderRadius: 8

Inline Styles

```
<View style={{ flex: 1, justifyContent: 'center' }}>
  <Text style={{ fontSize: 18, color: 'blue' }}>Inline Style</Text>
</View>
```

2.3 Flexbox Layout

Understanding Flexbox

React Native uses the same flexbox layout system as the web. Understanding flexbox is crucial for building responsive layouts .

Key Flex Properties

flex: 1 - Takes all available space :

```
<View style={{ flex: 1 }}>
  <View style={{ flex: 1, backgroundColor: 'red' }} /> { /* Takes 50% */}
  <View style={{ flex: 1, backgroundColor: 'blue' }} /> { /* Takes 50% */}
</View>
```

flexDirection - Direction of children:

```
<View style={{ flexDirection: 'row' }}>
  <View style={{ width: 50, height: 50, backgroundColor: 'red' }} />
  <View style={{ width: 50, height: 50, backgroundColor: 'blue' }} />
</View>
```

justifyContent - Alignment along main axis :

Value	Description
flex-start	Align at start
center	Center in middle
flex-end	Align at end
space-between	Even space between
space-around	Even space around

alignItems - Alignment along cross axis :

Value	Description
flex-start	Align at start
center	Center in middle
flex-end	Align at end
stretch	Stretch to fill

Flexbox Example: Card Layout

```
<View style={styles.card}>
  <Image source={{ uri: 'https://example.com/image.jpg' }} style={styles.cardImage} />
  <View style={styles.cardContent}>
    <Text style={styles.cardTitle}>Card Title</Text>
    <Text style={styles.cardDescription}>Card description goes here</Text>
    <View style={styles.cardFooter}>
      <Text style={styles.price}>$19.99</Text>
      <Button title="Buy" onPress={() => {}} />
    </View>
  </View>
</View>
```

```
const styles = StyleSheet.create({
  card: {
    backgroundColor: '#fff',
    borderRadius: 8,
    overflow: 'hidden',
    margin: 10,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.1,
    shadowRadius: 4,
    elevation: 3,
  },
});
```

```

    cardImage: {
      width: '100%',
      height: 200,
    },
    cardContent: {
      padding: 16,
    },
    cardTitle: {
      fontSize: 18,
      fontWeight: 'bold',
    },
    cardDescription: {
      fontSize: 14,
      color: '#666',
      marginTop: 4,
    },
    cardFooter: {
      flexDirection: 'row',
      justifyContent: 'space-between',
      alignItems: 'center',
      marginTop: 12,
    },
    price: {
      fontSize: 20,
      fontWeight: 'bold',
      color: '#007AFF',
    },
  },
},

```

2.4 Handling Touch Events

Button Component

The simplest way to handle taps:

```

import { Button } from 'react-native';

<Button
  title="Press Me"
  onPress={() => console.log('Pressed!')}
  color="#007AFF"
/>

```

Pressable Component

Pressable provides more control over touch interactions:

```
import { Pressable, Text } from 'react-native';

<Pressable
  onPress={() => console.log('Pressed')}
  onPressIn={() => console.log('Pressed In')}
  onPressOut={() => console.log('Pressed Out')}
  style={({ pressed }) => ({
    backgroundColor: pressed ? '#ddd' : '#007AFF',
    padding: 12,
    borderRadius: 8,
  })}
>
  <Text style={{ color: '#fff' }}>Press Me</Text>
</Pressable>
```

TouchableOpacity

For a button that fades on press:

```
import { TouchableOpacity, Text } from 'react-native';

<TouchableOpacity
  onPress={() => console.log('Pressed')}
  activeOpacity={0.7}
  style={styles.button}
>
  <Text style={styles.buttonText}>Tap Me</Text>
</TouchableOpacity>
```

TextInput Component

For user input:

```
import { TextInput } from 'react-native';

<TextInput
  style={styles.input}
  placeholder="Enter your name"
  value={text}
  onChangeText={setText}
  keyboardType="email-address" // or 'numeric', 'phone-pad'
  secureTextEntry={false}      // For passwords
  autoCapitalize="none"
/>
```

Day 2 Exercises

Exercise 2.1: Create a Profile Card

Instructions: Build a profile card component with:

1. An image (use a placeholder URL)
2. Name and title
3. Followers, following, posts counts
4. A "Follow" button

```
import React from 'react';
import { View, Text, Image, Button, StyleSheet } from 'react-native';

export default function ProfileCard() {
  // Your code here
}

const styles = StyleSheet.create({
  // Your styles here
});
```

Exercise 2.2: Responsive Layout with Flexbox

Instructions: Create a layout with:

1. A header at the top
2. Three columns in the middle
3. A footer at the bottom
4. Use flexbox for responsive layout

```
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';

export default function Dashboard() {
  // Your code here
}
```

Exercise 2.3: Interactive Form

Instructions: Build a simple registration form with:

1. Name input (text)
2. Email input (email keyboard)
3. Password input (secure text)
4. Submit button
5. Display submitted data

```
import React, { useState } from 'react';
import { View, Text, TextInput, Button, StyleSheet } from 'react-native';

export default function RegistrationForm() {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [submitted, setSubmitted] = useState(false);

  // Your code here
}
```

Day 2 Summary

Key Takeaways

1. **Core Components:** View , Text , Image , ScrollView , TextInput , Button
2. **StyleSheet:** CSS-like styling with camelCase properties
3. **Flexbox:** Layout system with flex , flexDirection , justifyContent , alignItems
4. **Touch Handling:** Button , Pressable , TouchableOpacity
5. **Forms:** TextInput with onChangeText and value

Quiz Yourself

1. What is the purpose of the View component?
2. How do you style components in React Native?
3. What is flexbox used for?
4. How do you handle user input in React Native?
5. What are the different ways to handle touch events?

Day 3: Lists, Forms & User Interactions

Learning Objectives

By the end of Day 3, you will be able to:

- Use FlatList for efficient list rendering
- Work with SectionList for grouped data
- Handle forms and form validation
- Implement pull-to-refresh and infinite scrolling
- Use AsyncStorage for persistent data

3.1 Lists in React Native

ScrollView vs FlatList

Feature	ScrollView	FlatList
Rendering	Renders all items at once	Renders only visible items
Performance	Slow for large lists	Optimized for large lists
Memory Usage	High	Low
Use Case	Small lists, known content	Large lists, dynamic data

When to use ScrollView: Small number of items (e.g., a settings page, form) **When to use FlatList:** Large or unknown number of items (e.g., feed, search results)

FlatList Component

```
import { FlatList, View, Text, StyleSheet } from 'react-native';

const DATA = [
  { id: '1', title: 'Item 1' },
  { id: '2', title: 'Item 2' },
  { id: '3', title: 'Item 3' },
];

<FlatList
  data={DATA}
  keyExtractor={({item}) => item.id}
  renderItem={({ item }) => (
    <View style={styles.item}>
      <Text style={styles.itemText}>{item.title}</Text>
    </View>
  )}
/>
```

```
    )}  
  />
```

FlatList with Custom Component

```
// src/components/ProductCard.js  
import React from 'react';  
import { View, Text, Image, StyleSheet } from 'react-native';  
  
export default function ProductCard({ product }) {  
  return (  
    <View style={styles.card}>  
      <Image source={{ uri: product.image }} style={styles.image} />  
      <View style={styles.content}>  
        <Text style={styles.name}>{product.name}</Text>  
        <Text style={styles.price}>${product.price}</Text>  
      </View>  
    </View>  
  );  
}  
  
// src/screens/ProductsScreen.js  
import React from 'react';  
import { FlatList, StyleSheet } from 'react-native';  
import ProductCard from '../components/ProductCard';  
  
const products = [  
  { id: '1', name: 'Product 1', price: 19.99, image: 'https://example.com/1.jpg' },  
  { id: '2', name: 'Product 2', price: 29.99, image: 'https://example.com/2.jpg' },  
];  
  
export default function ProductsScreen() {  
  return (  
    <FlatList  
      data={products}  
      keyExtractor={({ item }) => item.id}  
      renderItem={({ item }) => <ProductCard product={item} />  
      numColumns={2}  
      columnWrapperStyle={styles.row}  
      contentContainerStyle={styles.list}  
    />  
  );  
}  
  
const styles = StyleSheet.create({  
  row: { justifyContent: 'space-between' },  
  list: { padding: 16 },  
});
```

SectionList for Grouped Data

```
import { SectionList, Text, View, StyleSheet } from 'react-native';

const SECTIONS = [
  {
    title: 'Fruits',
    data: ['Apple', 'Banana', 'Cherry'],
  },
  {
    title: 'Vegetables',
    data: ['Carrot', 'Broccoli', 'Spinach'],
  },
];

<SectionList
  sections={SECTIONS}
  keyExtractor={({item, index}) => item + index}
  renderItem={({ item }) => (
    <Text style={styles.item}>{item}</Text>
  )}
  renderSectionHeader={({ section: { title } }) => (
    <Text style={styles.sectionHeader}>{title}</Text>
  )}
/>
```

3.2 Advanced List Features

Pull-to-Refresh

```
import React, { useState } from 'react';
import { FlatList, RefreshControl } from 'react-native';

export default function ProductsScreen() {
  const [refreshing, setRefreshing] = useState(false);
  const [data, setData] = useState([]);

  const onRefresh = async () => {
    setRefreshing(true);
    // Fetch new data
    const newData = await fetchData();
    setData(newData);
    setRefreshing(false);
  };

  return (
    <FlatList
```

```

    data={data}
    refreshControl={
      <RefreshControl
        refreshing={refreshing}
        onRefresh={onRefresh}
        colors={['#007AFF']}
      />
    }
    renderItem={({ item }) => <ProductCard product={item} />}
  />
);
}

```

Infinite Scrolling

```

import React, { useState } from 'react';
import { FlatList, ActivityIndicator } from 'react-native';

export default function ProductsScreen() {
  const [data, setData] = useState([]);
  const [loading, setLoading] = useState(false);
  const [hasMore, setHasMore] = useState(true);
  const [page, setPage] = useState(1);

  const loadMore = async () => {
    if (loading || !hasMore) return;

    setLoading(true);
    const newData = await fetchProducts(page);
    setData([...data, ...newData]);
    setHasMore(newData.length > 0);
    setPage(page + 1);
    setLoading(false);
  };

  return (
    <FlatList
      data={data}
      renderItem={({ item }) => <ProductCard product={item} />}
      onEndReached={loadMore}
      onEndReachedThreshold={0.5}
      ListFooterComponent={loading && <ActivityIndicator size="large" />}
    />
  );
}

```

3.3 Forms and Validation

Form Handling with State

```
import React, { useState } from 'react';
import { View, Text, TextInput, Button, StyleSheet, Alert } from 'react-native';

export default function LoginForm() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [errors, setErrors] = useState({});

  const validate = () => {
    const newErrors = {};
    if (!email) newErrors.email = 'Email is required';
    if (!email.includes('@')) newErrors.email = 'Invalid email';
    if (!password) newErrors.password = 'Password is required';
    if (password.length < 6) newErrors.password = 'Must be at least 6 characters';

    setErrors(newErrors);
    return Object.keys(newErrors).length === 0;
  };

  const handleSubmit = () => {
    if (validate()) {
      Alert.alert('Success', 'Form submitted!');
      console.log({ email, password });
    }
  };

  return (
    <View style={styles.container}>
      <View style={styles.field}>
        <Text style={styles.label}>Email</Text>
        <TextInput
          style={[styles.input, errors.email && styles.inputError]}
          value={email}
          onChangeText={setEmail}
          placeholder="Enter your email"
          keyboardType="email-address"
          autoCapitalize="none"
        />
        {errors.email && <Text style={styles.error}>{errors.email}</Text>}
      </View>

      <View style={styles.field}>
        <Text style={styles.label}>Password</Text>
        <TextInput
          style={[styles.input, errors.password && styles.inputError]}
          value={password}
          onChangeText={setPassword}
          placeholder="Enter your password"
          secureTextEntry
        />
      </View>
    </View>
  );
}
```

```
        {errors.password && <Text style={styles.error}>{errors.password}</Text>}
      </View>

      <Button title="Login" onPress={handleSubmit} />
    </View>
  );
}
```

3.4 Persistent Data with AsyncStorage

AsyncStorage Basics

AsyncStorage is a simple, unencrypted, asynchronous key-value storage system.

```
npm install @react-native-async-storage/async-storage
```

Basic Operations

```
import AsyncStorage from '@react-native-async-storage/async-storage';

// Save data
async function saveData(key, value) {
  try {
    await AsyncStorage.setItem(key, JSON.stringify(value));
  } catch (error) {
    console.error('Error saving data:', error);
  }
}

// Get data
async function getData(key) {
  try {
    const value = await AsyncStorage.getItem(key);
    return value ? JSON.parse(value) : null;
  } catch (error) {
    console.error('Error reading data:', error);
  }
}

// Remove data
async function removeData(key) {
  try {
    await AsyncStorage.removeItem(key);
  } catch (error) {
    console.error('Error removing data:', error);
  }
}
```

```
}  
}
```

Day 3 Exercises

Exercise 3.1: Product List with FlatList

Instructions: Build a product listing screen with:

1. FlatList with product data
2. Each product shows image, name, and price
3. Pull-to-refresh functionality
4. Two-column grid layout

Exercise 3.2: Login Form with Validation

Instructions: Build a login form with:

1. Email and password fields
2. Client-side validation
3. Error messages
4. Success alert on valid submission

Exercise 3.3: Notes App with AsyncStorage

Instructions: Build a simple notes app with:

1. Create, read, update, delete notes
2. AsyncStorage for persistence
3. FlatList for displaying notes

```
// Complete implementation  
import React, { useState, useEffect } from 'react';  
import {  
  View, Text, TextInput, Button, FlatList,  
  StyleSheet, Alert, TouchableOpacity  
} from 'react-native';  
import AsyncStorage from '@react-native-async-storage/async-storage';  
  
const STORAGE_KEY = '@notes';
```

```

export default function NotesApp() {
  const [notes, setNotes] = useState([]);
  const [title, setTitle] = useState('');
  const [content, setContent] = useState('');

  // Load notes on startup
  useEffect(() => {
    loadNotes();
  }, []);

  const loadNotes = async () => {
    const data = await AsyncStorage.getItem(STORAGE_KEY);
    if (data) {
      setNotes(JSON.parse(data));
    }
  };

  const saveNotes = async (newNotes) => {
    await AsyncStorage.setItem(STORAGE_KEY, JSON.stringify(newNotes));
    setNotes(newNotes);
  };

  const addNote = () => {
    if (!title || !content) {
      Alert.alert('Error', 'Please fill in all fields');
      return;
    }
    const newNote = {
      id: Date.now().toString(),
      title,
      content,
      createdAt: new Date().toISOString(),
    };
    saveNotes([...notes, newNote]);
    setTitle('');
    setContent('');
  };

  const deleteNote = (id) => {
    Alert.alert('Delete', 'Are you sure?', [
      { text: 'Cancel', style: 'cancel' },
      {
        text: 'Delete',
        style: 'destructive',
        onPress: () => {
          const filtered = notes.filter(note => note.id !== id);
          saveNotes(filtered);
        }
      }
    ], [
    ]);
  };

  return (

```

```

<View style={styles.container}>
  <Text style={styles.header}><img alt="pencil icon" data-bbox="418 53 438 68"/> Notes App</Text>

  <View style={styles.inputContainer}>
    <TextInput
      style={styles.input}
      placeholder="Note title"
      value={title}
      onChangeText={setTitle}
    />
    <TextInput
      style={[styles.input, styles.contentInput]}
      placeholder="Note content"
      value={content}
      onChangeText={setContent}
      multiline
    />
    <Button title="Add Note" onPress={addNote} />
  </View>

  <FlatList
    data={notes}
    keyExtractor={({item}) => item.id}
    renderItem={({item}) => (
      <TouchableOpacity
        style={styles.noteCard}
        onLongPress={() => deleteNote(item.id)}
      >
        <Text style={styles.noteTitle}>{item.title}</Text>
        <Text style={styles.noteContent} numberOfLines={2}>
          {item.content}
        </Text>
        <Text style={styles.noteDate}>
          {new Date(item.createdAt).toLocaleDateString()}
        </Text>
      </TouchableOpacity>
    )}
    ListEmptyComponent={
      <Text style={styles.emptyText}>No notes yet. Add one!</Text>
    }
  />
</View>
);
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    backgroundColor: '#f5f5f5',
  },
  header: {
    fontSize: 28,

```

```
    fontWeight: 'bold',
    marginBottom: 20,
  },
  inputContainer: {
    backgroundColor: '#fff',
    padding: 16,
    borderRadius: 8,
    marginBottom: 20,
  },
  input: {
    borderWidth: 1,
    borderColor: '#ddd',
    borderRadius: 4,
    padding: 12,
    marginBottom: 12,
  },
  contentInput: {
    minHeight: 80,
    textAlignVertical: 'top',
  },
  noteCard: {
    backgroundColor: '#fff',
    padding: 16,
    borderRadius: 8,
    marginBottom: 12,
    shadowColor: '#000',
    shadowOffset: { width: 0, height: 2 },
    shadowOpacity: 0.1,
    shadowRadius: 4,
    elevation: 2,
  },
  noteTitle: {
    fontSize: 18,
    fontWeight: 'bold',
  },
  noteContent: {
    color: '#666',
    marginTop: 4,
  },
  noteDate: {
    fontSize: 12,
    color: '#999',
    marginTop: 8,
  },
  emptyText: {
    textAlign: 'center',
    color: '#999',
    marginTop: 40,
  },
});
```

Day 3 Summary

Key Takeaways

1. **FlatList**: Efficient list rendering with `data`, `renderItem`, `keyExtractor`
2. **SectionList**: Grouped data with sections
3. **Pull-to-Refresh**: `RefreshControl` component
4. **Infinite Scrolling**: `onEndReached` and `onEndReachedThreshold`
5. **Forms**: State management with validation
6. **AsyncStorage**: Persistent key-value storage

Quiz Yourself

1. When should you use FlatList vs ScrollView?
2. How do you implement pull-to-refresh?
3. How do you handle form validation in React Native?
4. What is AsyncStorage used for?
5. How do you implement infinite scrolling?

Day 4: Navigation & State Management

Learning Objectives

By the end of Day 4, you will be able to:

- Implement navigation with React Navigation
- Use Stack Navigator for screen navigation
- Use Tab Navigator for bottom tabs
- Use Drawer Navigator for side menu
- Manage state with Context API
- Manage state with Redux Toolkit

4.1 React Navigation

Installation

```
npm install @react-navigation/native
npm install @react-navigation/native-stack
npm install @react-navigation/bottom-tabs
npm install @react-navigation/drawer
npm install react-native-screens react-native-safe-area-context
```

Navigation Container

```
// App.js
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';

const Stack = createNativeStackNavigator();

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen name="Home" component={HomeScreen} />
        <Stack.Screen name="Details" component={DetailsScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

Stack Navigator

```
// src/navigation/StackNavigator.js
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import HomeScreen from '../screens/HomeScreen';
import DetailsScreen from '../screens/DetailsScreen';
import ProfileScreen from '../screens/ProfileScreen';

const Stack = createNativeStackNavigator();

export default function StackNavigator() {
  return (
    <Stack.Navigator
      initialRouteName="Home"
      screenOptions={{
        headerStyle: {
          backgroundColor: '#007AFF',
        },
        headerTintColor: '#fff',
        headerTitleStyle: {
          fontWeight: 'bold',
        },
      },
    >
```

```

    }}
  >
  <Stack.Screen
    name="Home"
    component={HomeScreen}
    options={{ title: 'Home' }}
  />
  <Stack.Screen
    name="Details"
    component={DetailsScreen}
    options={({ route }) => ({
      title: route.params?.item?.name || 'Details'
    })}
  />
  <Stack.Screen
    name="Profile"
    component={ProfileScreen}
    options={{
      title: 'Profile',
      headerBackTitle: 'Back',
    }}
  />
</Stack.Navigator>
);
}

```

Navigation Between Screens

```

// src/screens/HomeScreen.js
import React from 'react';
import { View, Text, Button, StyleSheet } from 'react-native';

export default function HomeScreen({ navigation }) {
  return (
    <View style={styles.container}>
      <Text style={styles.title}>Home Screen</Text>

      <Button
        title="Go to Details"
        onPress={() => navigation.navigate('Details', {
          item: { id: 1, name: 'Item 1' }
        })}
      />

      <Button
        title="Go to Profile"
        onPress={() => navigation.navigate('Profile')}
      />

      <Button
        title="Go Back"

```

```
        onPress={() => navigation.goBack()}
      />
    </View>
  );
}
```

Passing Parameters

```
// Navigating with params
navigation.navigate('Details', {
  itemId: 86,
  itemName: 'Special Item',
});

// Receiving params
export default function DetailsScreen({ route }) {
  const { itemId, itemName } = route.params;
  return (
    <View>
      <Text>Item ID: {itemId}</Text>
      <Text>Item Name: {itemName}</Text>
    </View>
  );
}
```

4.2 Tab Navigator

```
// src/navigation/TabNavigator.js
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs';
import { Ionicons } from '@expo/vector-icons';
import HomeScreen from '../screens/HomeScreen';
import SettingsScreen from '../screens/SettingsScreen';
import ProfileScreen from '../screens/ProfileScreen';

const Tab = createBottomTabNavigator();

export default function TabNavigator() {
  return (
    <Tab.Navigator
      screenOptions={({ route }) => ({
        tabBarIcon: ({ focused, color, size }) => {
          let iconName;

          if (route.name === 'Home') {
            iconName = focused ? 'home' : 'home-outline';
          } else if (route.name === 'Settings') {
            iconName = focused ? 'settings' : 'settings-outline';
          }
        }
      })
    >
```

```

    } else if (route.name === 'Profile') {
      iconName = focused ? 'person' : 'person-outline';
    }

    return <Ionicons name={iconName} size={size} color={color} />;
  },
  tabBarActiveTintColor: '#007AFF',
  tabBarInactiveTintColor: 'gray',
  headerStyle: {
    backgroundColor: '#007AFF',
  },
  headerTintColor: '#fff',
}));
>
<Tab.Screen name="Home" component={HomeScreen} />
<Tab.Screen name="Settings" component={SettingsScreen} />
<Tab.Screen name="Profile" component={ProfileScreen} />
</Tab.Navigator>
);
}

```

4.3 Drawer Navigator

```

// src/navigation/DrawerNavigator.js
import { createDrawerNavigator } from '@react-navigation/drawer';
import { Ionicons } from '@expo/vector-icons';
import HomeScreen from '../screens/HomeScreen';
import AboutScreen from '../screens/AboutScreen';
import ContactScreen from '../screens/ContactScreen';

const Drawer = createDrawerNavigator();

export default function DrawerNavigator() {
  return (
    <Drawer.Navigator
      screenOptions={{
        drawerActiveTintColor: '#007AFF',
        drawerInactiveTintColor: '#666',
        drawerStyle: {
          backgroundColor: '#fff',
          width: 240,
        },
        headerStyle: {
          backgroundColor: '#007AFF',
        },
        headerTintColor: '#fff',
      }}
    >
      <Drawer.Screen

```

```

    name="Home"
    component={HomeScreen}
    options={{
      drawerIcon: ({ color }) => (
        <Ionicons name="home" size={24} color={color} />
      ),
    }}
  />
  <Drawer.Screen
    name="About"
    component={AboutScreen}
    options={{
      drawerIcon: ({ color }) => (
        <Ionicons name="information-circle" size={24} color={color} />
      ),
    }}
  />
  <Drawer.Screen
    name="Contact"
    component={ContactScreen}
    options={{
      drawerIcon: ({ color }) => (
        <Ionicons name="mail" size={24} color={color} />
      ),
    }}
  />
</Drawer.Navigator>
);
}

```

4.4 State Management

useState (Local State)

```

import React, { useState } from 'react';

export default function Counter() {
  const [count, setCount] = useState(0);
  return (
    <View>
      <Text>Count: {count}</Text>
      <Button title="Increment" onPress={() => setCount(count + 1)} />
    </View>
  );
}

```

useReducer (Complex State)

```
import React, { useReducer } from 'react';

const initialState = { count: 0 };

function reducer(state, action) {
  switch (action.type) {
    case 'increment':
      return { count: state.count + 1 };
    case 'decrement':
      return { count: state.count - 1 };
    case 'reset':
      return { count: 0 };
    default:
      return state;
  }
}

export default function Counter() {
  const [state, dispatch] = useReducer(reducer, initialState);
  return (
    <View>
      <Text>Count: {state.count}</Text>
      <Button title="+" onPress={() => dispatch({ type: 'increment' })} />
      <Button title="-" onPress={() => dispatch({ type: 'decrement' })} />
      <Button title="Reset" onPress={() => dispatch({ type: 'reset' })} />
    </View>
  );
}
```

Context API (Global State)

```
// src/context/AuthContext.js
import React, { createContext, useState, useContext } from 'react';

const AuthContext = createContext();

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);

  const login = (userData) => {
    setUser(userData);
  };

  const logout = () => {
    setUser(null);
  };

  return (
    <AuthContext.Provider value={{ user, login, logout }}>
      {children}
    </AuthContext.Provider>
  );
}
```

```
    </AuthContext.Provider>
  );
}

export function useAuth() {
  return useContext(AuthContext);
}

// Using the context
import React from 'react';
import { View, Text, Button } from 'react-native';
import { useAuth } from '../context/AuthContext';

export default function ProfileScreen() {
  const { user, logout } = useAuth();

  return (
    <View>
      <Text>Welcome, {user?.name}</Text>
      <Button title="Logout" onPress={logout} />
    </View>
  );
}
```

Day 4 Exercises

Exercise 4.1: Stack Navigation

Instructions: Build a simple app with three screens:

1. Home screen with a list of items
2. Details screen showing item details
3. Profile screen with user info
4. Implement navigation between screens

Exercise 4.2: Tab Navigation

Instructions: Add bottom tab navigation:

1. Home tab with dashboard
2. Search tab with search bar
3. Settings tab with options

Exercise 4.3: Drawer Navigation

Instructions: Add a drawer menu:

1. Home
2. About
3. Contact
4. Custom drawer styling

Exercise 4.4: Combined Navigation

Instructions: Build a complete navigation system:

1. Auth flow (Login/Register)
2. Main app with bottom tabs
3. Stack navigation within tabs

```
// Example combined structure
export default function App() {
  return (
    <NavigationContainer>
      <RootStack.Navigator>
        <RootStack.Screen
          name="Auth"
          component={AuthStack}
          options={{ headerShown: false }}
        />
        <RootStack.Screen
          name="Main"
          component={MainTabs}
          options={{ headerShown: false }}
        />
      </RootStack.Navigator>
    </NavigationContainer>
  );
}
```

Day 4 Summary

Key Takeaways

1. **React Navigation:** Core library for navigation
2. **Stack Navigator:** Screen stacking with header

3. **Tab Navigator:** Bottom tabs
4. **Drawer Navigator:** Side menu
5. **State Management:** useState, useReducer, Context API
6. **Navigation Flow:** Auth → Main app structure

Quiz Yourself

1. What is React Navigation?
2. How do you navigate between screens?
3. What is the difference between Stack and Tab navigation?
4. When would you use Context API?
5. What is useReducer used for?

Day 5: Native Features, Debugging & Publishing

Learning Objectives

By the end of Day 5, you will be able to:

- Access device features (camera, location)
- Implement push notifications
- Debug React Native apps
- Test apps on devices and emulators
- Publish apps to app stores

5.1 Native Device Features

Camera with Expo Camera

```
npx expo install expo-camera
```

```
import React, { useState, useRef } from 'react';
import { View, Text, TouchableOpacity, StyleSheet } from 'react-native';
import { Camera } from 'expo-camera';

export default function CameraScreen() {
```

```

const [hasPermission, setHasPermission] = useState(null);
const [type, setType] = useState(Camera.Constants.Type.back);
const cameraRef = useRef(null);

useEffect(() => {
  (async () => {
    const { status } = await Camera.requestCameraPermissionsAsync();
    setHasPermission(status === 'granted');
  })();
}, []);

if (hasPermission === null) {
  return <View />;
}

if (hasPermission === false) {
  return <Text>No access to camera</Text>;
}

const takePicture = async () => {
  if (cameraRef.current) {
    const photo = await cameraRef.current.takePictureAsync();
    console.log(photo.uri);
  }
};

return (
  <View style={styles.container}>
    <Camera
      ref={cameraRef}
      style={styles.camera}
      type={type}
    >
    <View style={styles.buttonContainer}>
      <TouchableOpacity
        style={styles.flipButton}
        onPress={() => {
          setType(
            type === Camera.Constants.Type.back
              ? Camera.Constants.Type.front
              : Camera.Constants.Type.back
          );
        }}
      >
        <Text style={styles.buttonText}>Flip</Text>
      </TouchableOpacity>

      <TouchableOpacity
        style={styles.captureButton}
        onPress={takePicture}
      >
        <View style={styles.captureCircle} />
      </TouchableOpacity>
    </View>
  </View>
);

```

```

    </View>
  </Camera>
</View>
);
}

```

Location

```
npx expo install expo-location
```

```

import React, { useState, useEffect } from 'react';
import { View, Text, Button, StyleSheet } from 'react-native';
import * as Location from 'expo-location';

export default function LocationScreen() {
  const [location, setLocation] = useState(null);
  const [errorMsg, setErrMsg] = useState(null);

  useEffect(() => {
    (async () => {
      const { status } = await Location.requestForegroundPermissionsAsync();
      if (status !== 'granted') {
        setErrMsg('Permission to access location was denied');
        return;
      }
    })();
  }, []);

  const getLocation = async () => {
    try {
      const currentLocation = await Location.getCurrentPositionAsync({});
      setLocation(currentLocation);
    } catch (error) {
      setErrMsg('Error getting location');
    }
  };

  return (
    <View style={styles.container}>
      <Text>Location App</Text>

      {errorMsg && <Text style={styles.error}>{errorMsg}</Text>}

      {location && (
        <View>
          <Text>Latitude: {location.coords.latitude}</Text>
          <Text>Longitude: {location.coords.longitude}</Text>
          <Text>Altitude: {location.coords.altitude}</Text>
        </View>
      )}
    </View>
  );
}

```

```

    )}

    <Button title="Get Location" onPress={getLocation} />
  </View>
);
}

```

Push Notifications

```
npx expo install expo-notifications
```

```

import React, { useState, useEffect } from 'react';
import { View, Text, Button, Platform } from 'react-native';
import * as Notifications from 'expo-notifications';

export default function NotificationsScreen() {
  useEffect(() => {
    registerForPushNotifications();
  }, []);

  async function registerForPushNotifications() {
    const { status } = await Notifications.requestPermissionsAsync();
    if (status !== 'granted') {
      alert('Failed to get push token for push notification!');
      return;
    }

    const token = await Notifications.getExpoPushTokenAsync();
    console.log('Push token:', token.data);
  }

  async function sendNotification() {
    await Notifications.scheduleNotificationAsync({
      content: {
        title: "Hello!",
        body: "This is a notification",
        data: { data: 'goes here' },
      },
      trigger: { seconds: 2 },
    });
  }

  return (
    <View style={styles.container}>
      <Text>Notifications</Text>
      <Button title="Send Notification" onPress={sendNotification} />
    </View>
  );
}

```

5.2 Debugging

React DevTools

The React DevTools extension helps inspect component hierarchy and state .

```
npm install -D react-devtools
```

Debugging Tools

Tool	Purpose
React DevTools	Component inspection
Chrome DevTools	JS debugging
React Native Debugger	Combined debugging
Expo Dev Tools	Expo-specific tools

Debugging Techniques

```
// 1. Console logging
console.log('State:', state);

// 2. Alert for quick messages
Alert.alert('Error', error.message);

// 3. React Native Debugger
// Install React Native Debugger for advanced debugging

// 4. Performance monitoring
console.time('render');
// Code to measure
console.timeEnd('render');
```

Debugging on Device

Android: Use `adb logcat` or Android Studio debugger **iOS:** Use Xcode debugger or Safari Web Inspector

5.3 Testing

Unit Testing with Jest

```
npm install --save-dev jest @testing-library/react-native
```

```
// __tests__/App.test.js
import React from 'react';
import { render, screen } from '@testing-library/react-native';
import App from '../App';

test('renders welcome text', () => {
  render(<App />);
  expect(screen.getByText(/Welcome to React Native/i)).toBeTruthy();
});
```

Component Testing

```
import React from 'react';
import { render, fireEvent } from '@testing-library/react-native';
import Button from '../Button';

test('button calls onPress when pressed', () => {
  const onPressMock = jest.fn();
  const { getByText } = render(
    <Button title="Press Me" onPress={onPressMock} />
  );

  fireEvent.press(getByText('Press Me'));
  expect(onPressMock).toHaveBeenCalledTimes(1);
});
```

5.4 Publishing Apps

Android Publishing

1. Build an APK/AAB:

```
npx expo run:android
# or
cd android
./gradlew assembleRelease
```

2. Sign the app:

- Generate a keystore
- Configure signing in `android/app/build.gradle`

3. Upload to Google Play Console:

- Create a developer account (\$25 one-time fee)
- Create a new application
- Upload the AAB file
- Fill in store listing
- Submit for review

iOS Publishing

1. Build the app:

```
npx expo run:ios  
# or in Xcode
```

2. Configure signing:

- Get an Apple Developer account (\$99/year)
- Create App ID and certificates
- Configure provisioning profiles

3. Upload to App Store Connect:

- Create a new app in App Store Connect
- Upload via Xcode or Transporter
- Fill in store listing
- Submit for review

Expo EAS Build (Recommended)

```
# Install EAS CLI  
npm install -g eas-cli  
  
# Login  
eas login  
  
# Configure build  
eas build:configure  
  
# Build for Android  
eas build --platform android
```

```
# Build for iOS
eas build --platform ios

# Submit to stores
eas submit --platform android
eas submit --platform ios
```

Day 5 Exercises

Exercise 5.1: Camera App

Instructions: Build a camera app with:

1. Camera preview
2. Flip camera button
3. Capture photo button
4. Display captured images

Exercise 5.2: Location Tracker

Instructions: Build a location tracker app:

1. Request location permission
2. Get current location
3. Display coordinates
4. Update location in real-time

Exercise 5.3: Debugging Practice

Instructions: Practice debugging:

1. Add console logs
2. Use React DevTools
3. Simulate errors and fix them

Exercise 5.4: Build and Publish

Instructions:

1. Build an APK for your app

2. Test the APK on a device
3. Prepare store listing

Day 5 Summary

Key Takeaways

1. **Native Features:** Camera, Location, Notifications via Expo
2. **Debugging:** Console, DevTools, React Native Debugger
3. **Testing:** Jest and React Native Testing Library
4. **Publishing:** Build APK/AAB, upload to stores
5. **EAS Build:** Simplified build and submission process

Quiz Yourself

1. How do you access the camera in React Native?
2. What tools can you use for debugging?
3. How do you test React Native components?
4. What is the process for publishing an Android app?
5. What is Expo EAS Build?

Final Assessment

React Native Mastery Test

Section 1: Core Concepts

Instructions: Answer the following questions.

1. What is React Native and what problem does it solve?
2. What is the difference between Expo and React Native CLI?
3. What are the basic building blocks of a React Native UI?
4. What is Flexbox and why is it important in React Native?
5. What is the difference between ScrollView and FlatList?

Section 2: Components

Instructions: Write the React Native code for the following.

1. Create a simple profile card component:

```
// Your code here
```

2. Create a list of items using FlatList:

```
// Your code here
```

3. Create a form with validation:

```
// Your code here
```

Section 3: Navigation

Instructions: Set up navigation for an app with the following screens:

- Home (Stack)
- Details (Stack)
- Settings (Tab)
- Profile (Tab)

```
// Your code here
```

Section 4: State Management

Instructions: Implement state management for a shopping cart:

1. Add items to cart
2. Remove items from cart
3. Show total items and price

```
// Your code here
```

Section 5: Native Features

Instructions: Write code to:

1. Request camera permission
2. Take a photo
3. Display the photo

```
// Your code here
```

Section 6: Complete Project

Instructions: Build a complete React Native app from scratch.

Requirements:

1. Authentication (login/register)
2. Home screen with feed
3. Profile screen
4. Settings with theme toggle
5. Navigation between screens
6. AsyncStorage for data persistence

Submit your code:

```
// Your complete app code here
```

React Native Quick Reference

Project Setup

```
# Expo
npx create-expo-app@latest MyApp
cd MyApp
npx expo start
```

```
# React Native CLI
npx react-native init MyApp
```

```
cd MyApp
npx react-native run-android
```

Core Components

```
import {
  View,
  Text,
  Image,
  ScrollView,
  FlatList,
  TextInput,
  Button,
  Pressable,
  TouchableOpacity,
  StyleSheet
} from 'react-native';
```

Styles

```
const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    backgroundColor: '#fff',
  },
  text: {
    fontSize: 16,
    fontWeight: 'bold',
    color: '#333',
  },
});
```

Navigation

```
npm install @react-navigation/native @react-navigation/native-stack
npm install react-native-screens react-native-safe-area-context
```

```
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';

const Stack = createNativeStackNavigator();

<NavigationContainer>
```

```
<Stack.Navigator>
  <Stack.Screen name="Home" component={HomeScreen} />
</Stack.Navigator>
</NavigationContainer>
```

State Management

```
// useState
const [state, setState] = useState(initialValue);

// Context API
const MyContext = createContext();

// useReducer
const [state, dispatch] = useReducer(reducer, initialState);
```

AsyncStorage

```
npm install @react-native-async-storage/async-storage
```

```
import AsyncStorage from '@react-native-async-storage/async-storage';

// Save
await AsyncStorage.setItem('key', JSON.stringify(value));

// Get
const value = await AsyncStorage.getItem('key');
```

Common Commands

Command	Description
<code>npx expo start</code>	Start Expo development
<code>npx expo run:android</code>	Run on Android
<code>npx expo run:ios</code>	Run on iOS
<code>npx expo build:android</code>	Build Android APK
<code>npx expo build:ios</code>	Build iOS app
<code>npx expo publish</code>	Publish updates

React Native Glossary

A

Android Studio: IDE for Android development, required for React Native CLI

APK: Android Package Kit - the installation file for Android apps

AsyncStorage: Key-value storage for persisting data in React Native

B

Bridge: Mechanism that connects JavaScript code to native components

Bundle: The compiled JavaScript file sent to the device

C

Component: Reusable building block in React Native

Cross-platform: Apps that run on multiple platforms (Android, iOS)

Core Components: Essential components like `View`, `Text`, `Image`

D

Drawer Navigator: Side menu navigation

DevTools: Tools for debugging and inspecting React Native apps

E

EAS: Expo Application Services - build and submission tools

Emulator: Android virtual device for testing

Expo: Framework for React Native development

F

FlatList: Efficient list rendering component

Flexbox: Layout system for arranging components

H

Hot Reloading: See changes in real-time during development

I

iOS Simulator: iOS virtual device for testing

J

JSX: JavaScript syntax extension for writing markup

N

Native Components: Platform-specific UI components

Navigation: Moving between screens in an app

P

Platform-specific: Code designed for a specific OS (Android/iOS)

Props: Properties passed to components

Pull-to-refresh: Swipe down gesture to refresh content

R

React Native: Framework for building native mobile apps

React Navigation: Library for navigation in React Native

Redux: State management library

RefreshControl: Component for pull-to-refresh

S

ScrollView: Scrollable container component

SectionList: List with grouped sections

Simulator: Virtual device for testing

Stack Navigator: Screen stacking navigation

State: Data that changes over time

StyleSheet: Styling system for React Native

T

Tab Navigator: Bottom tab navigation

TouchableOpacity: Component for press interactions

V

View: Basic container component

Course Completion Certificate

This certifies that

has successfully completed the

5-Day React Native Mobile App Development Course

covering all fundamental concepts including core components, styling, Flexbox layout, lists, navigation, state management, native features, debugging, and publishing apps to app stores.

Date: _____

Instructor: _____

Signature: _____